Komplexitätsanalyse

Einführung in die Programmierung 1
TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Algorithmen allgemein
- Motivierendes Beispiel

Was wird in diesem Foliensatz besprochen?

- Effizienzvergleich von Algorithmen
- Asymptotische Analyse

Was lernen Sie kennen?

- Unterschiedliche Herangehensweisen beim Vergleich von Effizienz
- Grundlagen der asymptotischen Analyse

Effizienz vergleichen

Messen

- Wie lange braucht ein Algorithmus für eine bestimmte Eingabe der Größe n (Laufzeit)?

Anzahl von Operationen zählen

- Wie viele Operationen führt ein Algorithmus für eine bestimmte Eingabe der Größe n aus?

Asymptotische Analyse

- Wie lässt sich die Laufzeit als Funktion der Eingabegröße n beschreiben?

Messen – maximale Abschnittssumme

Vergleich der Laufzeiten
• Hier Mittelwert von fünf Messungen pro Eingabegröße auf Rechner mit geringer Last • System: Intel Core i7 9750H, Windows 10, Java 14

Verdopplungsexperiment

• Eingabedaten werden immer verdoppelt
• Laufzeiten wachsen entsprechend an
  • Die Unterschiede in den Implementierungen zeigen sich erst bei großen Werten für *n*
  • Bei kleinen Werten dominieren andere Effekte, z. B. Caching

Verdopplungsexperiment

Eingabegröße n	1. Variante (sec)	2. Variante (sec)	3. Variante (sec)
1000	0.085	0.003	0
2000	0.661	0.003	0
4000	2.532	0.004	0
8000	20.089	0.008	0
16000	159.936	0.034	0
32000	1280	0.139	0
64000	10240	0.555	0
128000	81920	2.08	0
256000	655360	8.318	0
512000	5242880	33.214	0
1024000	41943040	133.285	0,001
2048000	335544320	532	0,002
4096000	2684354560 (ca. 85 Jahre)	2128	0,004

Doppelte Datenmenge führt zu ca. achtfacher Laufzeit!
Rote Werte extrapoliert (ungenau)

Doppelte Datenmenge führt zu ca. vierfacher Laufzeit!

Doppelte Datenmenge führt zu ca. doppelter Laufzeit!
Blaue Werte – nicht messbar/ungenau

Probleme mit Messungen

Beziehen sich auf eine Hardware/Software-Umgebung → Unterschiedliche Werte auf unterschiedlichen Umgebungen

Beziehen sich nur auf eine beschränkte Anzahl an Eingaben → Sind die verwendeten Eingaben (Eingabegrößen) repräsentativ?

Algorithmus muss implementiert werden, um seine Laufzeit zu messen

Ziele einer alternativen Analyse

- Evaluierung von Algorithmen unabhängig von der Hardware/Software-Umgebung
- Kann auch auf allgemeinere Beschreibungen (Pseudocode) angewendet werden, die noch keine konkreten Implementierungen darstellen
- Berücksichtigt alle möglichen Eingaben

Einfache Operationen zählen

Einfache Operationen

- Zuweisung
- Arithmetische Operation
- Vergleich
- Zugriff auf ein Element eines Arrays (über Index)
- Aufruf einer Methode bzw. Rückkehr aus einer Methode

Annahmen zur Ausführungszeit

- Konstant für eine spezifische einfache Operation
- Unterschied zwischen einzelnen Operationen gering (optimistisch bei Methodenaufrufen) und konstant

Beispiel (vereinfacht für Array mit n Elementen)

```
private static int maxSum3(int[] input) {
    int maxSum = input[0], endSum = maxSum;
    for (int pos = 1; pos < input.length; pos++) {
        endSum = Math.max(endSum + input[pos], input[pos]);
        maxSum = Math.max(maxSum, endSum);
    }
    return maxSum;
}
```

3 op

2 op

5 op

2 op

n-1 mal

1 op

Realistisch?
Aber: Genaue Anzahl pro Zeile nicht unbedingt wichtig.
Wichtig: Wie oft wird eine Zeile ausgeführt!

Operationen zählen

Ist unabhängig von der Hardware

Ist aber immer noch von der konkreten Implementierung abhängig

Es ist auch unklar, was alles gezählt werden soll

- Alle Operationen?
- Nur bestimmte?
- Nur die in der (innersten) Schleife?

Zählen für maxSum1 und maxSum2

Wie oft wird der Code (`sum += input[…]`) in der inneren Schleife für n Elemente ausgeführt?

n	maxSum1
10	220
20	1540
30	4960
40	11480
50	22100
100	171700
1000	167167000

n	maxSum2
10	55
20	210
30	465
40	820
50	1275
100	5050
1000	500500

Alternatives Beispiel

Verschiedene (aber gleich große) Inputs können das Verhalten eines Algorithmus beeinflussen

```
private static boolean search(int[] input, int element) {
    for (int i = 0; i < input.length; i++) {
        if (input[i] == element) {
            return true;
        }
    }
    return false;
}

public static void main(String[] args) {
    System.out.println(search(new int[]{1, 2, 3, 4, 5, 2, 3, 9}, 1));
    System.out.println(search(new int[]{1, 2, 3, 4, 5, 2, 3, 9}, 4));
    System.out.println(search(new int[]{1, 2, 3, 4, 5, 2, 3, 9}, 6));
}
```

Unterschiedliche Fälle – es wird nicht immer alles durchsucht

```
true
true
false
```

Unterschiedliche Fälle

Bester Fall (Best Case)	• Ist die Laufzeit eines Algorithmus bei der bestmöglichen Eingabe mit Größe **n**. • z. B. Suche: Gesuchter Wert befindet sich an erster Position im Array, d. h. nur ein Element muss untersucht werden.
Durchschnittlicher Fall (Average Case)	• Ist die durchschnittliche Laufzeit eines Algorithmus über alle möglichen gültigen Eingaben mit Größe **n**. • z. B. Suche: Bei Gleichverteilung ist der gesuchte Wert mit jeweils gleicher Wahrscheinlichkeit an jeder möglichen Stelle im Array. Im Durchschnitt werden ca. **n/2** Elemente untersucht.
Schlechtester Fall (Worst Case)	• Ist die größtmögliche Laufzeit eines Algorithmus bei einer Eingabe mit Größe **n**. • z. B. Suche: Gesuchter Wert befindet sich an letzter Position bzw. nicht im Array, d. h. alle **n** Elemente müssen untersucht werden.

Worst-Case

In der Praxis wird vom Worst-Case ausgegangen

Vorteile

- Meist einfacher den Worst-Case zu analysieren
- Führt zu besseren Algorithmen - Algorithmus muss auf allen Eingaben effizient ablaufen

Allgemeinerer Ansatz

Noch immer Operationen zählen

- Aber **allgemeiner**
- Kleine Variationen werden ignoriert
  - Z. B. ob 4, 5 oder 10 Operationen bei einer Iteration benötigt werden

Abhängigkeit von der Problemgröße

- Wie verhält sich der Algorithmus, wenn die Größe der Eingabe beliebig groß wird?

Größe der Eingabe muss genau beschrieben werden

- Z. B. eine einzige Zahl, Länge eines Arrays

Asymptotische Analyse – Ziele

Effizienz für große Eingabegrößen evaluieren

Wachstum der Laufzeit bei steigender Eingabegröße beschreiben

Möglichst enge obere Schranke für das Wachstum angeben

- Worst-Case berücksichtigen

Muss nicht ganz exakt sein

- Ordnung zählt
- Nur größte Faktoren bei der Laufzeit berücksichtigen

Asymptotische obere Schranke (O-Notation)

Es gilt für zwei Funktionen $f(n)$ und $g(n)$

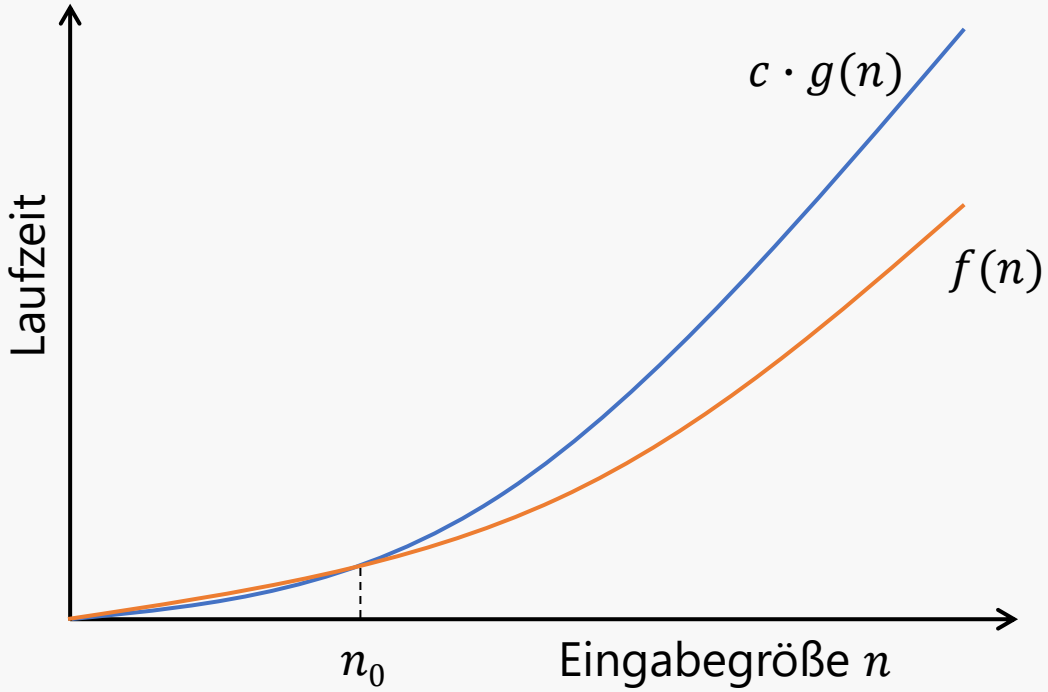
$$f(n)\ \ ist\ in\ \ O\big(g(n)\big)$$

Falls

$$\exists c > 0\ \exists n_0 > 0\ \forall n \geq n_0\ \ f(n) \leq c \cdot g(n)$$

Einschränkung: Wir betrachten Funktionen deren Definitionsbereich und Wertebereich die nicht-negativen ganzen Zahlen sind

Asymptotische obere Schranke



Laufzeit $f(n)$ mit Eingabegröße n und Schranke $c \cdot g(n)$ visualisiert

Notation

- Allgemein

$$f(n)\ \ ist\ in\ \ O\big(g(n)\big)$$

- Eigentlich

$$f(n) \in O\big(g(n)\big)$$

O(g(n)) bezeichnet die Menge aller Funktionen, die asymptotisch durch c•g(n) von oben beschränkt wird.

- Verkürzte (nicht ganz korrekte) Schreibweise

$$f(n) = O\big(g(n)\big)$$

Nicht als Gleichheit zu lesen!

Beispiele

$$8n + 5 \; ist \; in \; O(n)$$

Wir müssen Konstanten $c > 0$ und $n_0 > 0$ finden, sodass gilt

$$8n + 5 \; \leq c \cdot n \text{ für jedes } n \geq n_0$$

Mögliche Wahl: $c = 9$ und $n_0 = 5$

Es sind auch andere Werte möglich, solange diese eine korrekte obere Schranke ergeben.

n	8n + 5	9n
1	13	9
2	21	18
3	29	27
4	37	36
5	**45**	**45**
6	**53**	**54**
…		

Allgemein

Falls $f(n)$ ein Polynom vom Grad d ist

$$f(n) = a_0 + a_1 n + a_2 n^2 + \cdots + a_d n^d$$

und $a_d > 0$, dann gilt

$$f(n)\ ist\ in\ O(n^d)$$

Allgemein – Erklärung

Für $n \geq 1$ haben wir $1 \leq n \leq n^2 \leq \cdots \leq n^d$ und daher

$$a_0 + a_1 n + a_2 n^2 + \cdots + a_d n^d \leq (|a_0| + |a_1| + \cdots + |a_d|) n^d$$

Mit $c = |a_0| + |a_1| + \cdots + |a_d|$ und $n_0 = 1$ gilt $f(n)\ in\ O(n^d)$

Beispiele

$\boldsymbol{5n^4 + 3n^3 + 2n^2 + 4n + 1} \leq (5 + 3 + 2 + 4 + 1)n^4 = cn^4$ für $c = 15$ und $n_0 = 1$ daher $in\ \boldsymbol{O(n^4)}$

$\boldsymbol{5n^2 + 3n\,log\,n + 2n + 5} \leq (5 + 3 + 2 + 5)n^2 = cn^2$ für $c = 15$ und $n_0 = 1$ daher $in\ \boldsymbol{O(n^2)}$
Hinweis: $\log n \leq n$ für $n \geq 1$

$\boldsymbol{20n^3 + 10n\,log\,n + 5} \leq 35n^3$ für $n_0 = 1$ daher $in\ \boldsymbol{O(n^3)}$

Kürzere Variante

$\boldsymbol{2^{n+2}} = 2^n \cdot 2^2 = 4 \cdot 2^n$ für $c = 4$ und $n_0 = 1$ daher $in\ \boldsymbol{O(2^n)}$

Addition

$$O\big(f(n)\big) + O\big(g(n)\big)\ ist\ O\big(f(n) + g(n)\big)$$

Bei sequentiell aufeinanderfolgenden Teilen, z.B.

```
for (int i = 0; i < n; i++) {
    System.out.print("a");
}
for (int i = 0; i < n * n; i++) {
    System.out.print("b");
}
```

$O(n)$

$O(n * n)$

$$O(n) + O(n * n) = O(n + n^2) = O(n^2)$$

Multiplikation

$$O\big(f(n)\big) * O\big(g(n)\big) \; ist \; O\big(f(n) * g(n)\big)$$

Bei verschachtelten Teilen (Schleifen), die **beide von n** abhängen

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.print("a");
    }
}
```

$$O(n) * O(n) \; = \; O(n * n) \; = \; O(n^2)$$

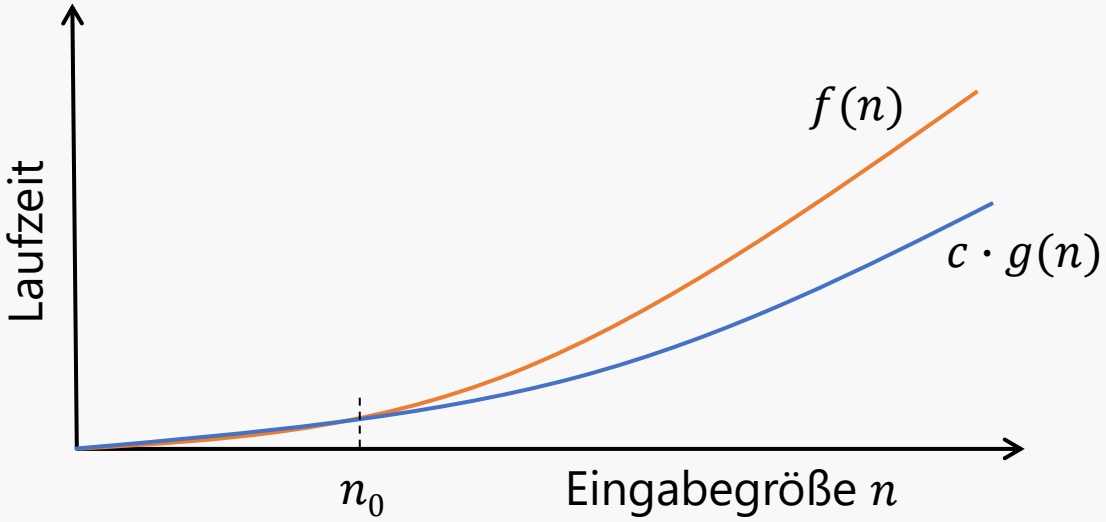
Weitere Schranken – untere Schranke

Es gilt für zwei Funktionen $f(n)$ und $g(n)$

$$f(n)\ \ ist\ in\ \ \Omega\big(g(n)\big)$$

Falls

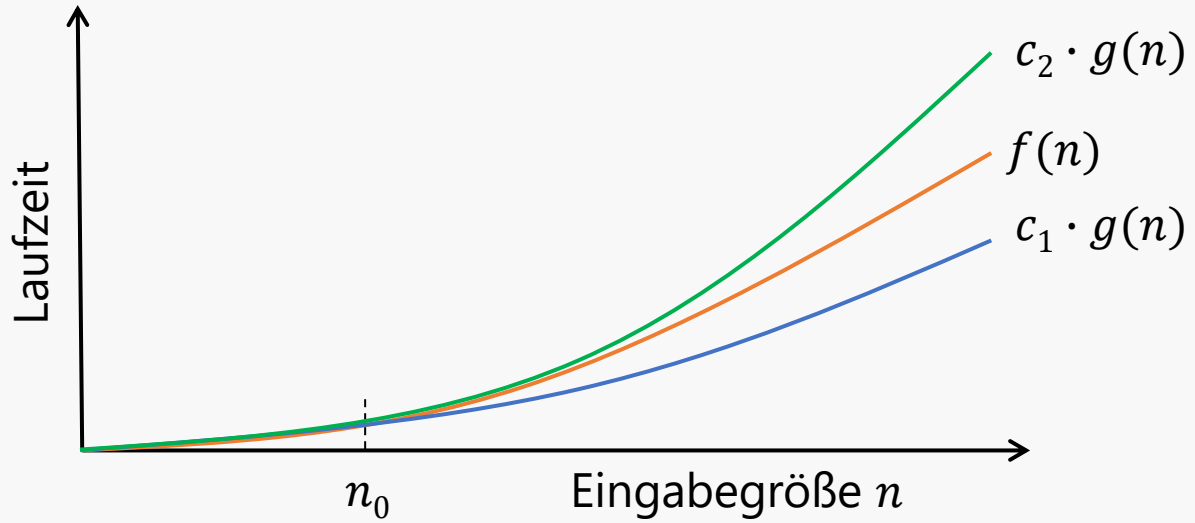
$$\exists c > 0\ \exists n_0 > 0\ \forall n \geq n_0\ \ c \cdot g(n) \leq f(n)$$



Weitere Schranken – scharfe Schranke

Es gilt für zwei Funktionen $f(n)$ und $g(n)$

$$f(n) \;\; ist\ in \;\; \Theta\big(g(n)\big)$$

Falls

$$\exists c_1 > 0 \;\; \exists c_2 > 0 \;\; \exists n_0 > 0 \;\; \forall n \geq n_0 \quad c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$